



Dataplug



Unlocking extreme data analytics with on-the-fly dynamic partitioning of unstructured data

A. Arjona, P. García-López and D. Barcelona-Pons, "Dataplug: Unlocking extreme data analytics with on-the-fly dynamic partitioning of unstructured data," *2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, Philadelphia, PA, USA, 2024

EXTRACT

A distributed data-mining software platform for extreme data across the compute continuum



**UNIVERSITAT
ROVIRA I VIRGILI**



This work has been partially funded by the European Union through Horizon Europe projects



Who are we?

Cloud and Distributed Systems Lab



- 4 active large-scale EU-funded projects (>€1M each, 3 coordinated)



- Top-tier publications (Middleware, CCGrid, ICDCS, USENIX ATC)

- Strong collaborations with elite partners



- Not only research but also production-grade solutions



Pedro
García

Enrique
Molina



+ 25 members

Contents

I - Context and problem statement

II - Dataplug: on-the-fly data partitioning

III - Use cases

IV - Use case validation

V - Conclusion

I – Problem Statement

Extreme data repositories



Extreme data → Extreme volume, speed, complexity and variety in values

- Huge repositories of **unstructured scientific data** from different domains (e.g. genomics, metabolomics, ...) and many different **data formats**
- Challenge current distributed cloud computing technologies

THE CANCER
GENOME ATLAS



- Genomics Open Data repository → ~9 PB
- Variety of data formats (BAM, VCF...)
- Variety of size (from few KB to TB)
- Distributed through Amazon S3 object storage

Unstructured data partitioning



Cloud Environment

- ❖ “*Infinite*” amount of on-demand elastic compute and storage resources
- ❖ **Object storage** as data source (e.g. Amazon S3)



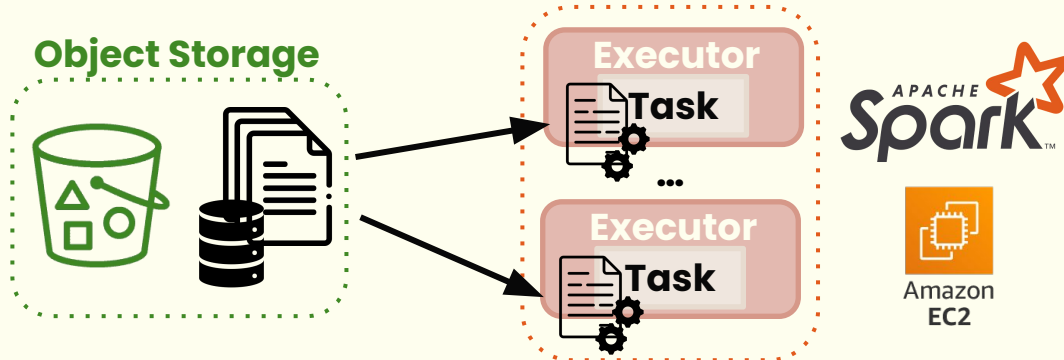
I – Problem Statement

Unstructured data partitioning

Cloud Environment

- ❖ “*Infinite*” amount of on-demand elastic compute and storage resources
- ❖ **Object storage** as data source (e.g. Amazon S3)

Fine-grained data partitioning is key to fully exploit the Cloud’s scalability and parallelism → Large datasets are chunked into smaller individual logical partitions, processed in parallel (data-parallel workloads)



E.g. In Spark, a large dataset is partitioned into multiple individual files that are processed by one task each



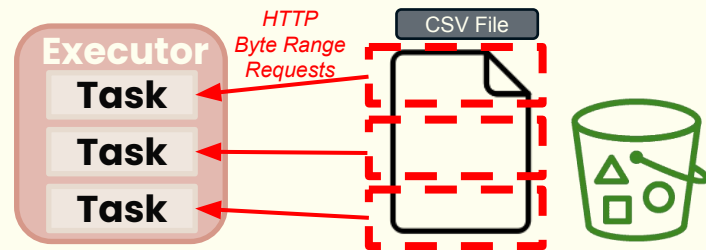
I – Problem Statement

Partial Reads from Object Storage

Object storage allows to read partial content in parallel using *HTTP Byte-range* requests

- Generic tabular or array-like data formats
→ CSV, JSON, AVRO, Zarr... *semi-structured data!*

 `spark.read.csv("path/to/data.csv")`



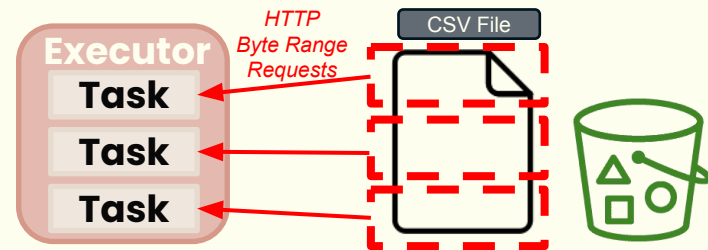
Partial Reads from Object Storage



Object storage allows to read partial content in parallel using *HTTP Byte-range* requests

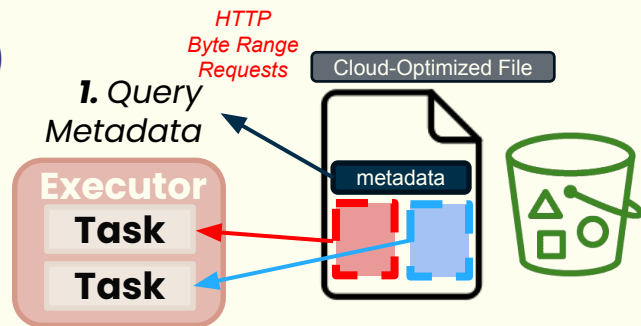
- Generic tabular or array-like data formats
→ CSV, JSON, AVRO, Zarr... *semi-structured data!*

 `spark.read.csv("path/to/data.csv")`



- ❖ Cloud Optimized File Formats
(e.g. Cloud Optimized GeoTIFF for satellite imaging)

- Stores **embedded** metadata in file headers → Allow to **query** file contents
→ Problem: Data has to be converted to Cloud-Optimized
→ Objects are **immutable** → Rewrite entire object



Unstructured data partitioning



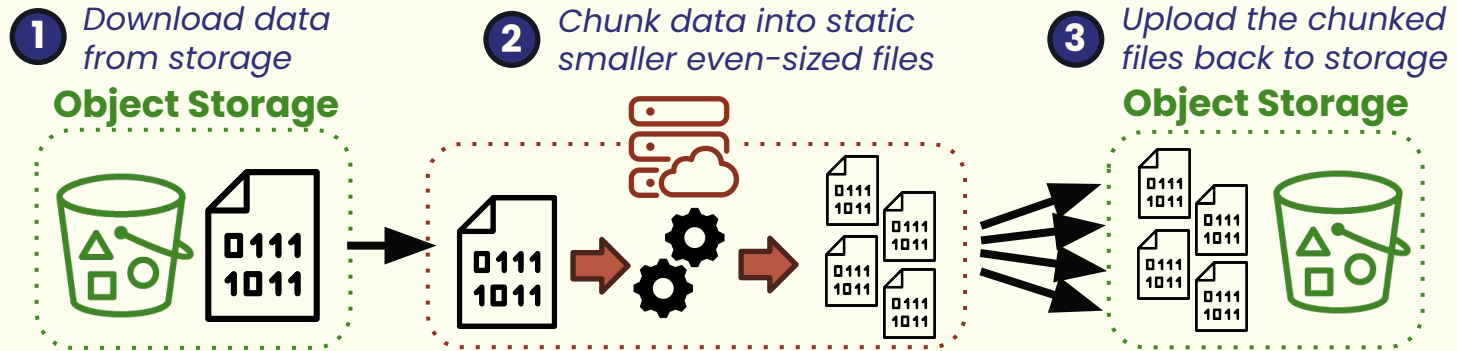
How to partition large datasets of *unstructured* scientific data formats?
→ **Static partitioning**



I – Problem Statement

Unstructured data partitioning

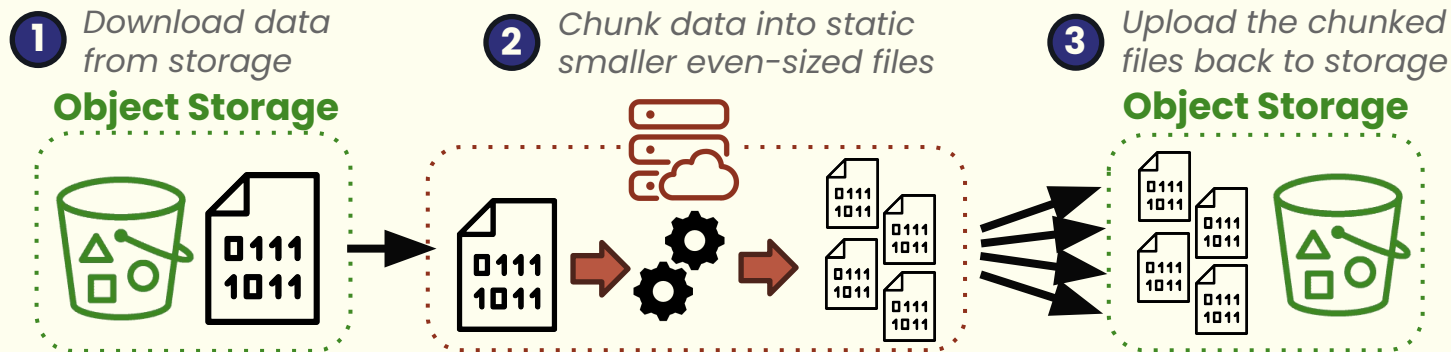
How to partition large datasets of *unstructured* scientific data formats?
→ **Static partitioning**





How to partition large datasets of *unstructured* scientific data formats?

→ **Static partitioning**



Cloud distributed computing frameworks allow only to process **individual unstructured files**



```
spark.read.format("binaryFile").load("/path/to/data")
```

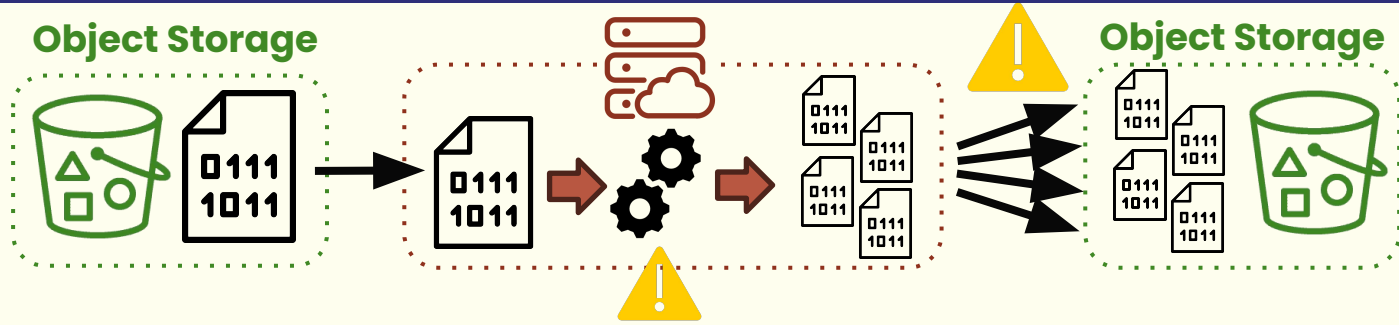


```
sample, blocks = dask.bytes.read_bytes('s3://bucket/data/*',  
                                       block_size='128 MiB')
```



I – Problem Statement

Unstructured data partitioning



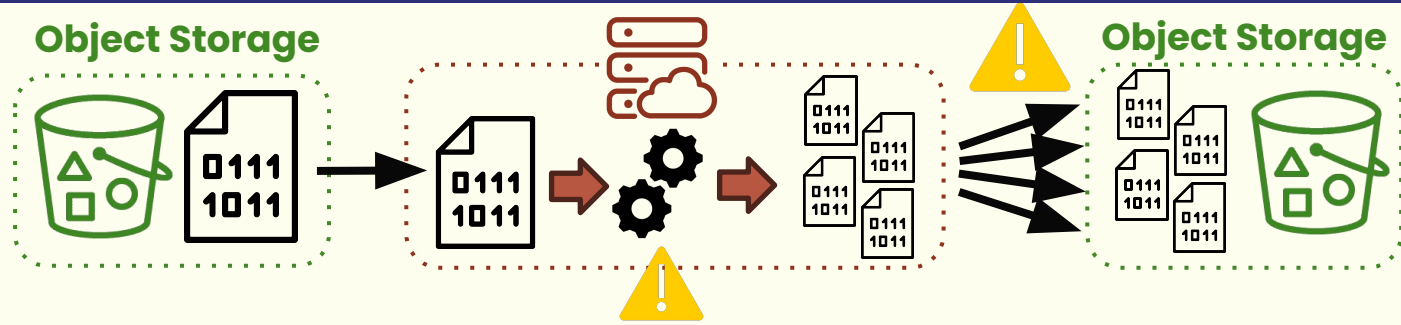
Static partitioning is not an ideal approach for *extreme data*:

- Processing for chunking files is challenging → Variety in format & size
- Huge data movements (whole dataset moved twice → download & upload)
- Other issues: Duplicated data in storage? Chunk size? Number of partitions?



I – Problem Statement

Unstructured data partitioning



Static partitioning is not an ideal approach for *extreme data*:

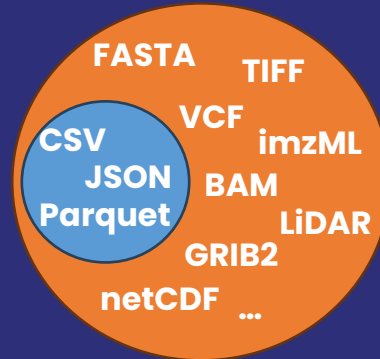
- Processing for chunking files is challenging → Variety in format & size
- Huge data movements (whole dataset moved twice → download & upload)
- Other issues: Duplicated data in storage? Chunk size? Number of partitions?

As a result, many **unstructured scientific data repositories** remain *untapped* due to the complexity they present to be processed efficiently in parallel



Problem Statement Summary

- **Data partitioning is key** to efficiently process scientific data in the cloud
- **Static partitioning is not ideal** for large unstructured datasets
- Object storage → **Partial reads** at **high bandwidth** but data that cannot be **accessed in parallel** poses a **challenge for scalability**
- **Cloud-Optimized** formats are **not enough**: require transformation and only allows to query (not partitioning)



-   Structured data
-   Unstructured (scientific) data 

Contents

I - Context and problem statement

II - Dataplug: on-the-fly data partitioning

III - Use cases

IV - Use case validation

V - Conclusion

II – Dataplug: on-the-fly data partitioning

Dataplug



Contribution: Novel data management model to enable efficient on-the-fly dynamic partitioning of large unstructured data blobs in cloud object storage → Alternative to *static partitioning*

II – Dataplug: on-the-fly data partitioning

Dataplug



Contribution: Novel data management model to enable efficient on-the-fly dynamic partitioning of large unstructured data blobs in cloud object storage → Alternative to *static partitioning*

- The framework: **Dataplug** that implements the *on-the-fly* data partitioning model
- Enable efficient **parallel access** to existing unstructured data blobs in their original scientific format
- **Hide the complexities** of scientific data pre-processing and partitioning → User-friendly, portable (Dask, Spark, ...)
- **Extensible** plugin interface to support many scientific file formats from different domains → OSS community

Dataplug's data management model:

1 Cloud-aware read-only preprocessing



which enables...

2 Dynamic on-the-fly partitioning

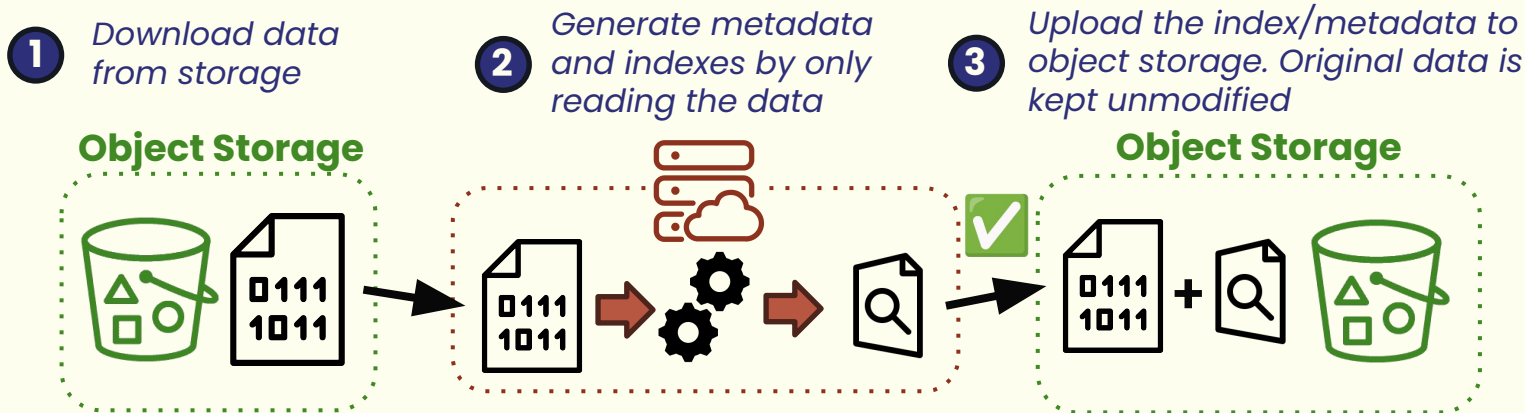
II – Dataplug: on-the-fly data partitioning

Cloud-aware pre-processing



→ To enable parallel data access to unstructured data → **Pre-process**

Dataplug approach → **Generate reusable index and metadata from cold unstructured data by only reading it**



→ Each data format supported has its own preprocessing/indexing logic
Extensible → Plugin interface to support new formats

II – Dataplug: on-the-fly data partitioning

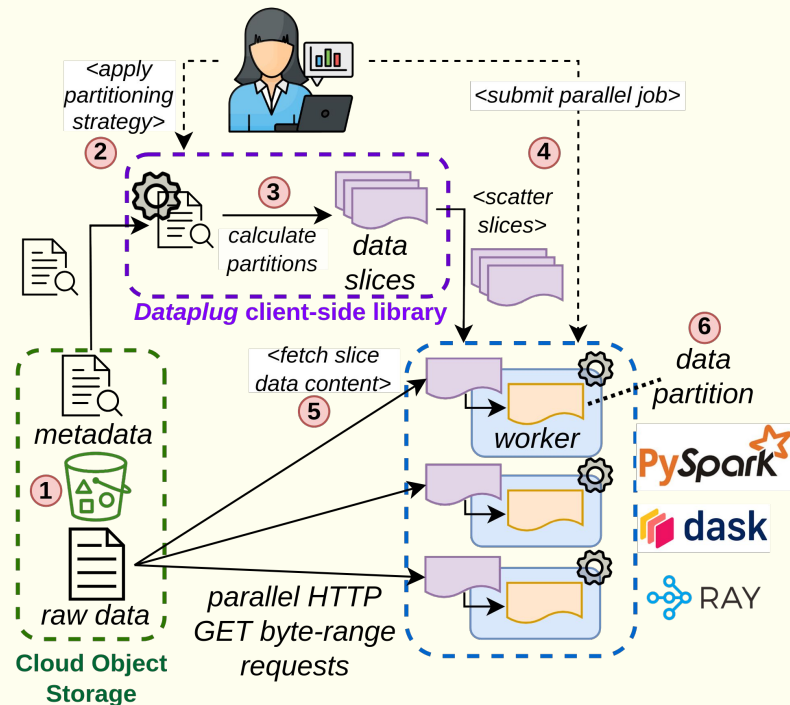
Dynamic on-the-fly partitioning



After preprocessing → Data blobs can be accessed in parallel

Data slicing: Dynamic on-the-fly partitioning of a dataset → Partitioning at **runtime**

Data slice → software entity, lazily-evaluated partition of an unstructured data blob



Contents

I - Context and problem statement

II - Dataplug: on-the-fly data partitioning

III - Use cases

IV - Use case validation

V - Conclusion

LiDAR Point Cloud Geospatial Data

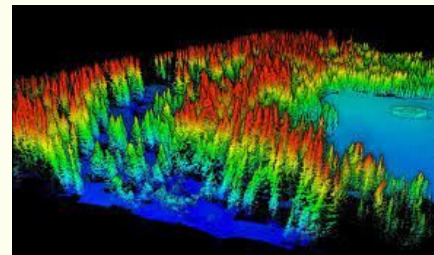
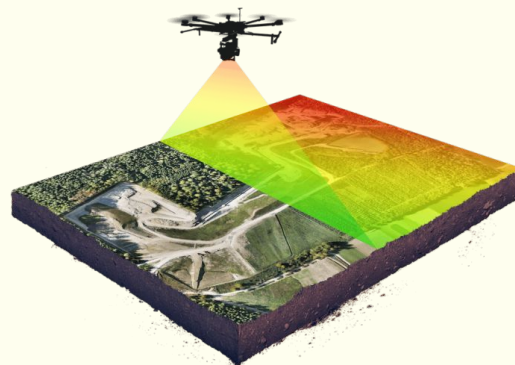


→ **LiDAR** → Binary format for 3-dimensional point cloud data that represents the Earth's surface morphology

Challenge → Points are scattered across the file, arranged by stripes (drone sweeps). Filter all the points to find those belonging to a subregion.

Solution → Preprocess with LAS tooling to generate point index. The points are indexed by region in a tree structure. Byte ranges are stored for each tree node.

Cloud-aware LiDAR → Parallel read of geographical subregions. Partitioning strategies with logic to partition by geographical sub-regions.



- **FASTQGZip** → Compressed text-based format to represent genome sequence read data and their quality values

Challenge → GZip does not allow to decompress arbitrary portions. Requires to de-compress the whole file from the beginning.

Solution → Preprocess with GZip tooling to generate GZip index. Allows to decompress from multiple entry points indexed by line number.

Cloud-aware FASTQGZip → Parallel decompression of chunks. Partitioning strategies with logic to preserve FASTQ format (e.g. partition by no. of sequences)

	Header	Sequence	Quality
@HWI-ST227:389:C4WA2ACXX:7:1204:2272:59979	GAGGAAAGGTCCCTCGCTCCTCTTTCATATAAGGGAAATGGCTGAAT	+	FFFFHHHHHHJJJJJJJJJJJJJIGIGIGIJJJJIJJJJJJII
@HWI-ST227:389:C4WA2ACXX:7:1205:15214:42893	GAGGATCCCAAGGAGGAAGGCTCGCTCCTCTTCATCATAGGGA	+	12BAFB?A:3<AE1<<FF;1?*EG*)?0?DBD>9BF9B*?#####
@HWI-ST227:389:C4WA2ACXX:8:2208:2467:44624	AAAGAGGAGAGAGGACCATTCTCCCTGGGATCCTCAGAAGTCTACT	+	BDDA:DB?2AA@FC>F?EEGC<FED>GFD;?GBB?<?F99*/9?9?

Measurement Set Astronomical Data

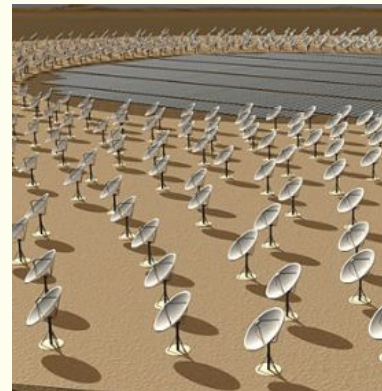


→ **Measurement Set** → Format (directory: set of files) used in radio astronomy to store interferometric observation data.

Challenge → Time-series data scattered across different files in the directory format. There are some common data and particular data.

Solution → Identify the common and particular objects in the file. Index the files that need to be chunked in the partition operation.

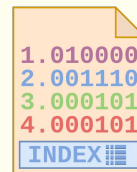
Cloud-aware Measurement Set → Parallel read of different temporal windows and common data.



Cloud-optimized and structured formats



The Dataplug API is also applicable to
Cloud-Optimized and tabular (structured)
data



Cloud-
Optimized
Data Blob

- COG → Cloud-Optimized GeoTIFF
- COPC → Cloud-Optimized Point Cloud
- CSV files



COPC.io

Cloud Optimized Point Cloud



COG

CLOUD OPTIMIZED
GEOTIFF

III – Use cases

How can I use it?



```
from dataplug import CloudObject
from dataplug.formats.genomics.fastq import FASTQGZip, partition_reads_batches

# Assign FASTQGZip data type for object in s3://genomics/SRR6052133_1.fastq.gz
co = CloudObject.from_s3(FASTQGZip, "s3://genomics/SRR6052133_1.fastq.gz")

# Data must be pre-processed first ==> This only needs to be done once per dataset
# Preprocessing will create reusable indexes to repartition
# the data many times in many chunk sizes
# Dataplug leverages joblib to deploy preprocessing jobs
co.preprocess(parallel_config={"backend": "dask"})

# Partition the FASTQGZip object into 200 chunks
# This does not move data around, it only creates data slices from the indexes
data_slices = co.partition(partition_reads_batches, num_batches=200)

def process_fastq_data(data_slice):
    # Evaluate the data_slice, which will perform the
    # actual HTTP GET requests to get the FASTQ partition data
    fastq_reads = data_slice.get()
    ...

# Use Dask for deploying a parallel distributed job
import dask.bag as db
from dask.distributed import Client

client = Client()
# Create a Dask Bag from the data_slices list
dask_bag = db.from_sequence(data_slices)

# Apply the process_fastq_data function to each data slice
# Dask will serialize the data_slices and send them to the workers
dask_bag.map(process_fastq_data).compute()
```

III – Use cases

How can I use it?

```
from dataplug import CloudObject
from dataplug.formats.genomics.fastq import FASTQGZip, partition_reads_batches

# Assign FASTQGZip data type for object in s3://genomics/SRR6052133_1.fastq.gz
co = CloudObject.from_s3(FASTQGZip, "s3://genomics/SRR6052133_1.fastq.gz")

# Data must be pre-processed first ==> This only needs to be done once per dataset
# Preprocessing will create reusable indexes to repartition
# the data many times in many chunk sizes
# Dataplug leverages joblib to deploy preprocessing jobs
co.preprocess(parallel_config={"backend": "dask"})

# Partition the FASTQGZip object into 200 chunks
# This does not move data around, it only creates data slices from the indexes
data_slices = co.partition(partition_reads_batches, num_batches=200)

def process_fastq_data(data_slice):
    # Evaluate the data_slice, which will perform the
    # actual HTTP GET requests to get the FASTQ partition data
    fastq_reads = data_slice.get()
    ...

# Use Dask for deploying a parallel distributed job
import dask.bag as db
from dask.distributed import Client

client = Client()
# Create a Dask Bag from the data_slices list
dask_bag = db.from_sequence(data_slices)

# Apply the process_fastq_data function to each data slice
# Dask will serialize the data_slices and send them to the workers
dask_bag.map(process_fastq_data).compute()
```

III – Use cases

How can I use it?

```
from dataplug import CloudObject
from dataplug.formats.genomics.fastq import FASTQGZip, partition_reads_batches

# Assign FASTQGZip data type for object in s3://genomics/SRR6052133_1.fastq.gz
co = CloudObject.from_s3(FASTQGZip, "s3://genomics/SRR6052133_1.fastq.gz")

# Data must be pre-processed first ==> This only needs to be done once per dataset
# Preprocessing will create reusable indexes to repartition
# the data many times in many chunk sizes
# Dataplug leverages joblib to deploy preprocessing jobs
co.preprocess(parallel_config={"backend": "dask"})

# Partition the FASTQGZip object into 200 chunks
# This does not move data around, it only creates data slices from the indexes
data_slices = co.partition(partition_reads_batches, num_batches=200)

def process_fastq_data(data_slice):
    # Evaluate the data_slice, which will perform the
    # actual HTTP GET requests to get the FASTQ partition data
    fastq_reads = data_slice.get()
    ...

# Use Dask for deploying a parallel distributed job
import dask.bag as db
from dask.distributed import Client

client = Client()
# Create a Dask Bag from the data_slices list
dask_bag = db.from_sequence(data_slices)

# Apply the process_fastq_data function to each data slice
# Dask will serialize the data_slices and send them to the workers
dask_bag.map(process_fastq_data).compute()
```

III – Use cases

How can I use it?

```
from dataplug import CloudObject
from dataplug.formats.genomics.fastq import FASTQGZip, partition_reads_batches

# Assign FASTQGZip data type for object in s3://genomics/SRR6052133_1.fastq.gz
co = CloudObject.from_s3(FASTQGZip, "s3://genomics/SRR6052133_1.fastq.gz")

# Data must be pre-processed first ==> This only needs to be done once per dataset
# Preprocessing will create reusable indexes to repartition
# the data many times in many chunk sizes
# Dataplug leverages joblib to deploy preprocessing jobs
co.preprocess(parallel_config={"backend": "dask"})

# Partition the FASTQGZip object into 200 chunks
# This does not move data around, it only creates data slices from the indexes
data_slices = co.partition(partition_reads_batches, num_batches=200)

def process_fastq_data(data_slice):
    # Evaluate the data_slice, which will perform the
    # actual HTTP GET requests to get the FASTQ partition data
    fastq_reads = data_slice.get()
    ...

# Use Dask for deploying a parallel distributed job
import dask.bag as db
from dask.distributed import Client

client = Client()
# Create a Dask Bag from the data_slices list
dask_bag = db.from_sequence(data_slices)

# Apply the process_fastq_data function to each data slice
# Dask will serialize the data_slices and send them to the workers
dask_bag.map(process_fastq_data).compute()
```

III – Use cases

How can I use it?

```
from dataplug import CloudObject
from dataplug.formats.genomics.fastq import FASTQGZip, partition_reads_batches

# Assign FASTQGZip data type for object in s3://genomics/SRR6052133_1.fastq.gz
co = CloudObject.from_s3(FASTQGZip, "s3://genomics/SRR6052133_1.fastq.gz")

# Data must be pre-processed first ==> This only needs to be done once per dataset
# Preprocessing will create reusable indexes to repartition
# the data many times in many chunk sizes
# Dataplug leverages joblib to deploy preprocessing jobs
co.preprocess(parallel_config={"backend": "dask"})

# Partition the FASTQGZip object into 200 chunks
# This does not move data around, it only creates data slices from the indexes
data_slices = co.partition(partition_reads_batches, num_batches=200)

def process_fastq_data(data_slice):
    # Evaluate the data_slice, which will perform the
    # actual HTTP GET requests to get the FASTQ partition data
    fastq_reads = data_slice.get()
    ...

# Use Dask for deploying a parallel distributed job
import dask.bag as db
from dask.distributed import Client

client = Client()
# Create a Dask Bag from the data_slices list
dask_bag = db.from_sequence(data_slices)

# Apply the process_fastq_data function to each data slice
# Dask will serialize the data_slices and send them to the workers
dask_bag.map(process_fastq_data).compute()
```

Contents

I - Context and problem statement

II - Dataplug: on-the-fly data partitioning

III - Use cases

IV - Use case validation

V - Conclusion

IV – Use cases validation

Validation Goals



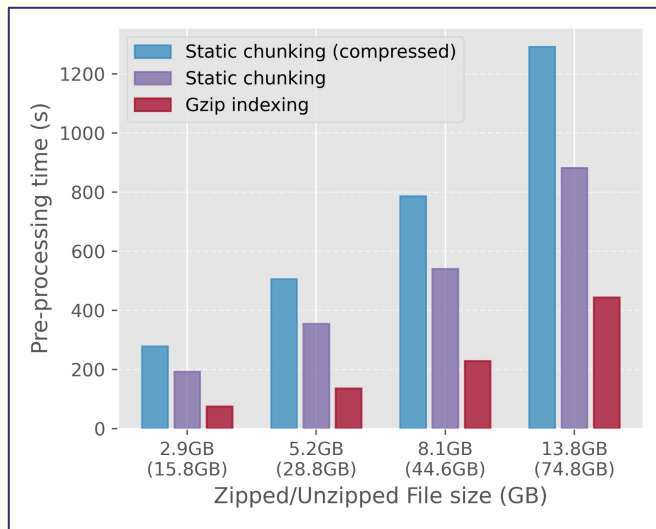
- Experiments → Cloud-aware **FASTQGZip** and **LiDAR**
 - ◆ Cloud-aware pre-processing **reduces compute time and storage costs** without introducing overhead
 - ◆ Demonstrate the **benefits of dynamic on-the-fly partitioning**
 - (i) Avoid in-cluster partitioning
 - (ii) Allow zero-cost re-partitioning to probe for the optimal partition size

IV – Use cases validation

Cloud-aware FASTQGzip Performance



- Compare between **Static partitioning** (using `aws-cli`, `zcat` and `split` commands) and **Cloud-aware preprocessing** (using Dataplug)



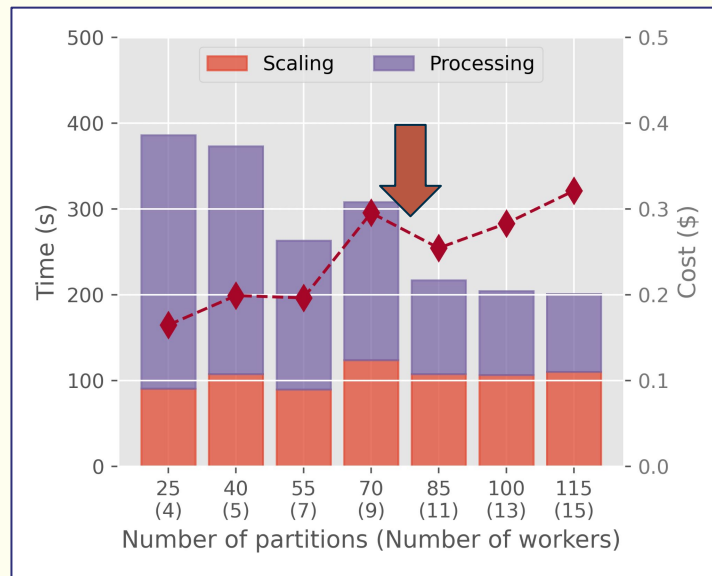
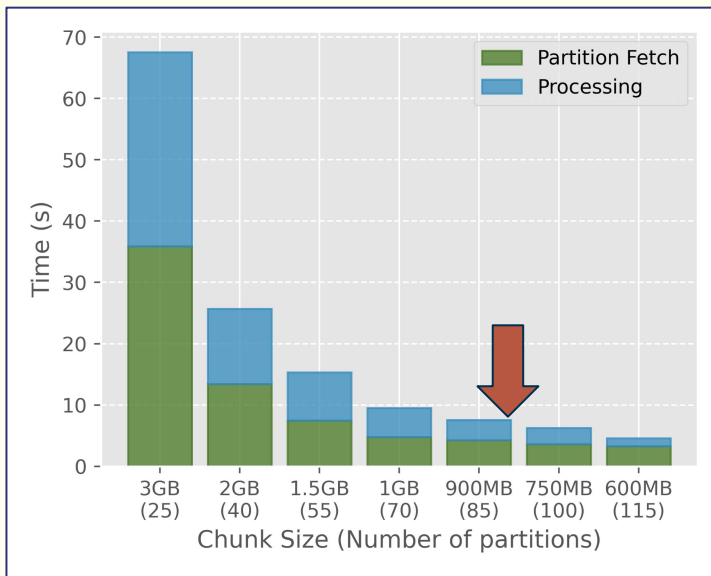
- *Pre-processing time is ~65% faster using Cloud-aware FASTQ (Dataplug)*



IV – Use cases validation

FASTQGzip Chunk Size Probing

- With dynamic partitioning → **Re-partitioning a dataset is zero-cost**
- Data-parallel FASTQ Genomics Workload with Dask on Amazon EC2 → **Probe different chunk sizes FASTQ data to find the most optimal one**
- Deploy full workload (75 GB file) to check guessed chunk size

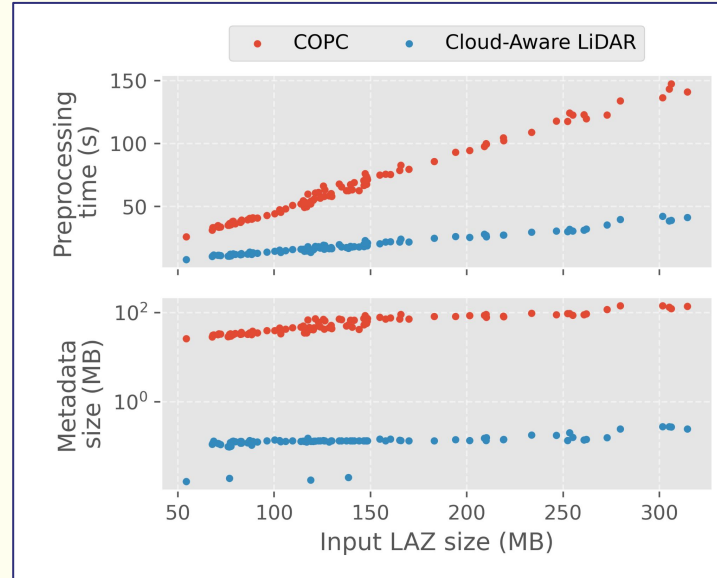




IV – Use cases validation

Cloud-Aware LiDAR Performance

- Compare between transforming LiDAR to **Cloud-Optimized Point Cloud (COPC)** and **Pre-processing Cloud-Aware LiDAR (Dataplug)**

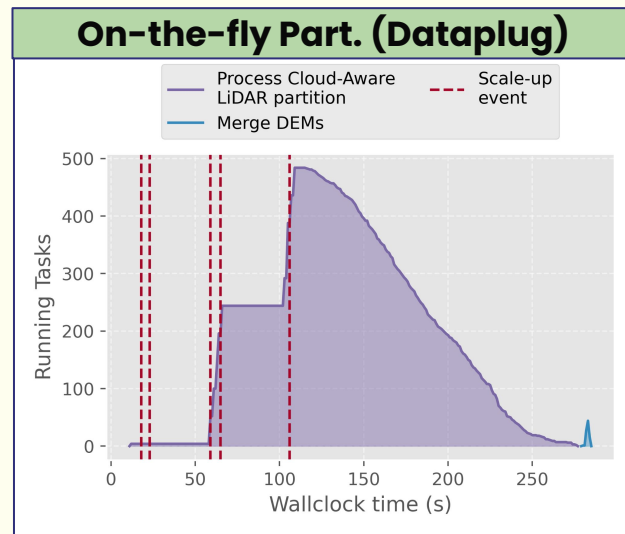
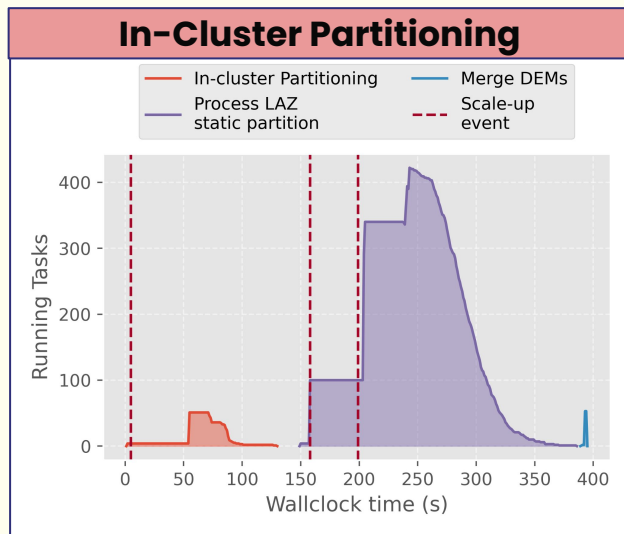


- *Cloud-Aware LiDAR reduction of ~71% in pre-processing time and 99.8% reduction in metadata volume size compared to COPC*

On-the-fly LiDAR Partitioning in Autoscaling Clusters



- With on-the-fly partitioning → Partitions are fetched directly from object storage → Avoid in-cluster re-partitioning
- Data-parallel LiDAR Geospatial workload with an Auto-scaling Ray Cluster in Amazon EC2 (53 GB dataset)



- Scale-up events are issued earlier → ~28% reduction in runtime



Contents

I - Context and problem statement

II - Dataplug: on-the-fly data partitioning

III - Use cases

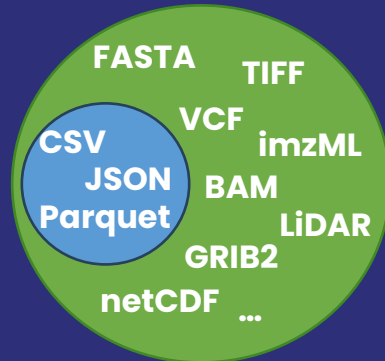
IV - Use case validation

V - Conclusion



Takeaways

- **Dataplug**: Dynamic on-the-fly partitioning of scientific data in the cloud
- Dataplug enables parallel access to **unstructured scientific datasets efficiently**
- Key point → Avoid rewriting data when doing **static partitioning**
- Demonstrate benefits with two representative formats (more already supported)



Structured data



Unstructured
(scientific) data



Dataplug current status

Dataplug was an
accepted research
paper in **CCGRID'24**

<https://ieeexplore.ieee.org/document/10701348>

2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)

Dataplug: Unlocking extreme data analytics with on-the-fly dynamic partitioning of unstructured data

Albert Arjona
Universitat Rovira i Virgili
Tarragona, Spain
alor.arjona@urv.cat

Pedro García-López
Universitat Rovira i Virgili
Tarragona, Spain
pedro.garcia@urv.cat

Daniel Barcelona-Pons
Universitat Rovira i Virgili
Tarragona, Spain
daniel.barcelona@urv.cat

Abstract—The elasticity of the Cloud is very appealing for processing large scientific data. However, enormous volumes of unstructured research data, totaling petabytes, remain untapped in data repositories due to the lack of efficient parallel data access. Event-driven partitioning of these data to enable its parallel processing requires a complex re-write to storage, becoming prohibitively expensive for high volumes. In this article we present Dataplug, an extensible framework that enables fine-grained parallel data access in unstructured scientific data to object storage. Dataplug employs read-only, format-aware indexing, allowing to define dynamically-sized partitions using various partitioning strategies. This approach avoids writing the partitioned dataset back to storage, enabling distributed workers to fetch data partitions on-the-fly directly from large data blobs, efficiently leveraging the high bandwidth capability of object storage. Validation on generic (FASTQ/ZIP) and genomics (LHAK) data formats demonstrate that Dataplug considerably lowers pre-processing compute costs between 62.5% – 73.2% (vs) without imposing significant overheads.

Index Terms—data partitioning, Cloud, object storage, scientific data management, unstructured data, data analytics

I. INTRODUCTION

The seemingly “infinite” compute and storage resources available in the Cloud have paved the way for handling unprecedented amounts of unstructured data, marking the evolution from big data to what is termed as *extreme data* [1]. Extreme data is characterized by its increasing volume, speed, variety, complexity and extreme variations in values, which

we can distribute the workload to process each sub-region separately, increasing the overall parallelism.

However, partitioning extreme unstructured data at this scale is challenging. A common approach for unstructured data is to partition a dataset into smaller event-sized files or objects, or transform it into a “Cloud-friendly” format [6]. This, however, involves reading, pre-processing, and writing back to storage the whole dataset, which can become extremely costly. Moreover, this issue is further exacerbated in Cloud scenarios, where object storage serves as the primary storage for scientific computing in the Cloud [6]. Since objects are immutable, partitioning a dataset into many objects requires re-writing all the data, making this approach inefficient. Finally, arbitrarily setting a fixed partition size without any hints can result in suboptimal performance [4], given the variation in workload requirements. While distributed computing frameworks like Dask or Spark can partition large files in arbitrary byte ranges, this approach is only applicable to generic formats like CSV and it breaks unstructured data formats.

Thus, we argue that static partitioning is not a viable solution for extreme scientific computing. Instead, we advocate for an alternative approach involving read-only format-aware pre-processing enabling on-the-fly dynamic partitioning.

Read-only format-aware pre-processing generates indexes and metadata for existing cold unstructured data without overwriting it. This approach eliminates the need to write back the





Dataplug current status

UNIVERSITAT

Dataplug was an
accepted research
paper in **CCGRID'24**

<https://ieeexplore.ieee.org/document/10701348>

Dataplug is an
open-source Python
library

<https://github.com/CLOUDLAB-URV/dataplug>

2024 IEEE 34th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)

Dataplug: Unlocking extreme data analytics with on-the-fly dynamic partitioning of unstructured data

Albert Arjona
Universitat Rovira i Virgili
Tarragona, Spain
albert.arjona@urv.cat

Pedro García-López
Universitat Rovira i Virgili
Tarragona, Spain
pedro.garcia@urv.cat

Daniel Barcelona-Pons
Universitat Rovira i Virgili
Tarragona, Spain
daniel.barcelona@urv.cat

Abstract—The elasticity of the Cloud is very appealing for processing large scientific data. However, enormous volumes of unstructured research data, totaling petabytes, remain untapped in data repositories due to the lack of efficient parallel data access. Event-driven partitioning of these data to enable its parallel processing requires a complex re-write to storage, becoming prohibitively expensive for high volumes. In this article we present Dataplug, an extensible framework that enables fine-grained parallel data access to unstructured scientific data as object storage. Dataplug employs read-only, format-aware indexing, allowing to define dynamically-sized partitions using various partitioning strategies. This approach avoids writing the partitioned dataset back to storage, enabling distributed workers to fetch data partitions on-the-fly directly from large data blobs, efficiently leveraging the high bandwidth capability of object storage. Validating on generic (FASTQ) and genomics (LARK) data formats demonstrate that Dataplug considerably lowers pre-processing compute costs (between 62.5% – 73.2%) w/o without imposing significant overheads.

Index Terms—data partitioning, Cloud, object storage, scientific data management, unstructured data, data analytics

I. INTRODUCTION

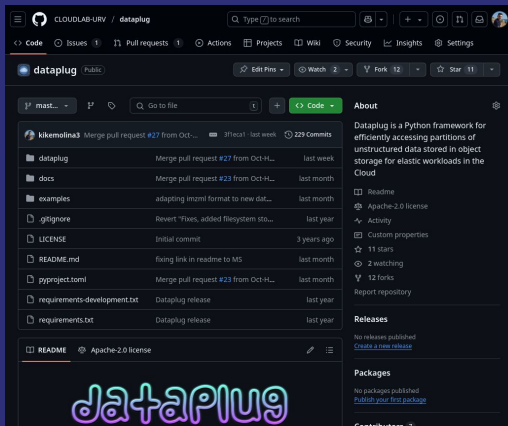
The seemingly “infinite” compute and storage resources available in the Cloud have paved the way for handling unprecedented amounts of unstructured data, marking the evolution from big data to what is termed as *extreme data* [1]. Extreme data is characterized by its increasing volume, speed, variety, complexity and diverse varieties. In science, which

we can distribute the workload to process each sub-region separately, increasing the overall parallelism.

However, partitioning extreme unstructured data at this scale is challenging. A common approach for unstructured data is to partition a dataset into smaller event-sized files or objects, or transform it into a “Cloud-friendly” format [6]. This, however, involves reading, pre-processing, and writing back to storage the whole dataset, which can become extremely costly. Moreover, this issue is further exacerbated in Cloud scenarios, where object storage serves as the primary storage for scientific computing in the Cloud [6]. Since objects are immutable, partitioning a dataset into many objects requires re-writing all of the data, making this approach inefficient. Finally, arbitrarily setting a fixed partition size without any hints can result in suboptimal performance [4], given the variation in workload requirements. While distributed computing frameworks like Dask or Spark can partition large files in arbitrary byte ranges, this approach is only applicable to generic formats like CSV and it breaks unstructured data formats.

Thus, we argue that static partitioning is not a viable solution for extreme scientific computing. Instead, we advocate for an alternative approach involving read-only format-aware pre-processing enabling on-the-fly dynamic partitioning.

Read-only format-aware pre-processing generates indexes and metadata for existing cold unstructured data without overwriting it. This approach eliminates the need to write back the





Dataplug current status

UNIVERSITAT

Dataplug was an
accepted research
paper in **CCGRID'24**

<https://ieeexplore.ieee.org/document/10701348>

Dataplug is an
open-source Python
library

<https://github.com/CLOUDLAB-URV/dataplug>

Call to action

→ Use it!
→ More formats

Dataplug: Unlocking extreme data analytics with on-the-fly dynamic partitioning of unstructured data

Albert Arjona
Universitat Rovira i Virgili
Tarragona, Spain
alor.arjona@urv.cat

Pedro García-López
Universitat Rovira i Virgili
Tarragona, Spain
pedro.garcia@urv.cat

Daniel Barcelona-Pons
Universitat Rovira i Virgili
Tarragona, Spain
daniel.barcelona@urv.cat

Abstract—The elasticity of the Cloud is very appealing for processing large scientific data. However, enormous volumes of unstructured research data, totaling petabytes, remain untapped in data repositories due to the lack of efficient parallel data access. Event-driven partitioning of these data to enable its parallel processing requires a complex re-write to storage, becoming prohibitively expensive for high volumes. In this article we present *Dataplug*, an extensible framework that enables fine-grained parallel data access to unstructured scientific data in object storage. *Dataplug* employs read-only, format-aware indexing, allowing to define dynamically-sized partitions using various partitioning strategies. This approach avoids writing the partitioned dataset back to storage, enabling distributed workers to fetch data partitions on-the-fly directly from large data blobs, efficiently leveraging the high bandwidth capability of object storage. Validating on generic HADOOP-like and genomics (LARK) data formats demonstrate that *Dataplug* considerably lowers pre-processing compute costs between 62.5% — 73.2% w/o without imposing significant overheads.

Index Terms—data partitioning, Cloud, object storage, scientific data management, unstructured data, data analytics

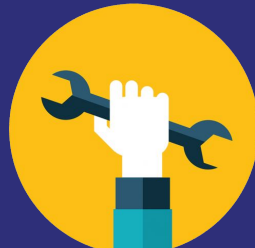
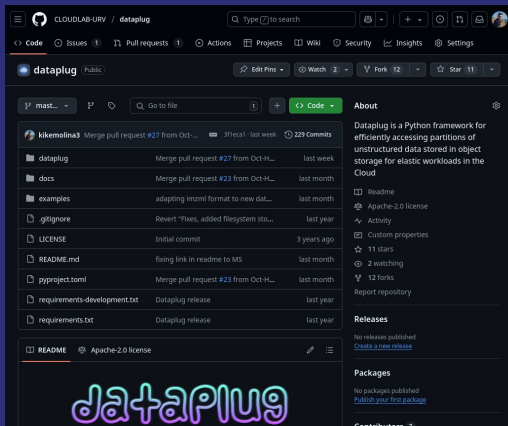
I. INTRODUCTION

The seemingly “infinite” compute and storage resources available in the Cloud have paved the way for handling unprecedented amounts of unstructured data, marking the evolution from big data to what is termed as *extreme data* [1]. Extreme data is characterized by its increasing volume, speed, variety, complexity and extreme variations in values, which

we can distribute the workload to process each sub-region separately, increasing the overall parallelism.

However, partitioning extreme unstructured data at this scale is challenging. A common approach for unstructured data is to partition a dataset into smaller event-sized files or objects, or transform it into a “Cloud-friendly” format [6]. This, however, involves reading, pre-processing, and writing back to storage the whole dataset, which can become extremely costly. Moreover, this issue is further exacerbated in Cloud scenarios, where object storage serves as the primary storage for scientific computing in the Cloud [6]. Since objects are immutable, partitioning a dataset into many objects requires re-writing all the data, making this approach inefficient. Finally, arbitrarily setting a fixed partition size without any hints can result in suboptimal performance [4], given the variation in workload requirements. While distributed computing frameworks like Dask or Spark can partition large files in arbitrary byte ranges, this approach is only applicable to generic formats like CSV and it breaks unstructured data formats.

Thus, we argue that static partitioning is not a viable solution for extreme scientific computing. Instead, we advocate for an alternative approach involving read-only format-aware pre-processing enabling on-the-fly dynamic partitioning. Read-only format-aware pre-processing generates indexes and metadata for existing cold unstructured data without overwriting it. This approach eliminates the need to write back the



Thank you!

Any questions?



This work has been partially funded by the European Union through the Horizon Europe EXTRACT (101093110) project.

EXTRACT

A distributed data-mining software platform for extreme data across the compute continuum

extract-project.eu